

An enterprise Azure estate you can delete on purpose

Meridian Launch Systems is a fictional launch provider whose entire Azure environment is defined, deployed, verified and destroyed from one repository. The repository is the product. The cloud estate is a build artifact.

Audience: engineering, security & operations leadership Synthetic data throughout

11 LAYERS, EACH INDEPENDENTLY AUDITED	43 VERIFICATION CRITERIA	1,428 AUTOMATED TESTS	< 1 hr DESTROY & REBUILD
< \$5 MONTHLY IDLE COST	1 MANUAL STEP REMAINING		

LENS 01 **Process — how the environment is governed**

The engineering practices are the point. Every one of them is a control an auditor, a finance partner, or a security officer can inspect without taking anyone's word for it.

The repository is the source of truth

Nothing is configured by hand in a portal. Anything that exists in the estate can be recreated by replaying pipelines — which means configuration drift becomes visible instead of accumulating quietly. Exactly one manual step survives, and it is documented as an exception rather than glossed over.

Verification is independent, and it is code

Each layer ships an audit script that queries Azure, Entra, Fabric and GitHub directly under a *separate read-only identity* – never the identity that performed the deployment. A layer is complete when an independent auditor confirms the deployed state matches intent, not when a deployment exits successfully. The audits are read-only by construction: their transport wrappers refuse any mutating verb.

Human approval sits only where it earns its cost

Five gates, placed where money, credentials, or irreversibility are involved. Everywhere else, independent verification substitutes for routine review – so approval means something instead of becoming a reflex.

Reproducibility is treated as a control, not a convenience

The environment is destroyed and rebuilt deliberately. A rebuild that succeeds proves the repository is complete; a rebuild that fails names precisely what was missing. This is disaster recovery, onboarding, and audit evidence produced by the same mechanism.

Deployment is layered and propagation-aware

Identity objects, policy assignments and label replication settle on their own schedules. The build order respects that, with bounded retry windows per criterion, so the estate is never declared ready while it is still converging.

Cost discipline is enforced, not encouraged

A tag taxonomy is enforced by Azure Policy – untagged resource groups are refused at creation. Daily cost exports land in the analytics plane, and spend per application over time is a first-class operational view rather than a monthly spreadsheet.

G0	Bootstrap	Tenant, subscription, root federation, capacity. Human-only, one time.
G1	Plan approval	The full build plan approved once. No per-layer sign-off thereafter.
G2	Spend profile	Anything that raises the run rate, with the projected delta stated before approval.
G3	Tenant-level deletion	Identity objects, labels, workspaces. Resource-group teardown is deliberately gate-free.

G4

Exception

Repeated verification failure, cost anomaly, or credential failure. Otherwise silence.

LENS 02

Technology — what the engineering demonstrates

Microsoft-native and standards-based throughout, with the security properties designed in rather than bolted on.

No stored Azure credentials

Every pipeline authenticates to Azure by workload identity federation; every service authenticates to Azure data planes by managed identity. The Azure SQL server enforces Entra-only authentication, so no database password exists to leak. Nothing authenticating to Azure is a stored secret, and the repository itself holds none at all — a full-history scanner fails the build on any finding.

Six long-lived credentials do exist, and naming them is the point: two Entra app certificates, because Security & Compliance PowerShell has no federated path; two GitHub tokens, because a `GITHUB_TOKEN` push does not trigger workflows; and two values in Key Vault, read at run time. None of them authenticates to Azure, none is in the repository, and all six are listed in the incident-response rotation text. An earlier revision of this page said continuous integration “holds no secret at all”. That was false, it is recorded as finding F28, and it is corrected here rather than quietly dropped.

Infrastructure on Azure Verified Modules

Microsoft-maintained, version-pinned Bicep modules rather than hand-rolled templates. Where no verified module fits, the raw resource is used deliberately and the reason recorded alongside it.

Governance expressed as policy	Management groups, tag enforcement, permitted regions, and the NIST SP 800-53 initiative assigned as code. Compliance posture is queryable, versioned, and rebuilt with everything else.
The AI agent is version-controlled infrastructure	The Copilot Studio agent is a Power Platform solution exported into the repository and deployed by pipeline — not a thing someone built in a browser. Editing it in a portal causes the layer audit to fail, which is the intended behaviour.
The agent cannot generate its own interface	It answers with declarative component specifications — validated against a schema before they are returned — which a fixed renderer draws. That is a governance property: the surface an executive sees cannot be produced by a language model at runtime.
Tools are exposed over an open protocol	Operational, security and cost tools are published through Model Context Protocol, so the same tool surface serves the agent and the dashboards. Each tool is read-only and allow-listed; anything outside the list is refused before execution, not after.
Software supply chain covered end to end	SPDX software bills of materials, static analysis, dependency scanning, container image scanning, dynamic application testing, secret scanning with push protection, and full-history secret sweeps — each with a negative test proving the gate actually blocks.
Remediation runs without a human in the path	A vulnerability finding produces a fix, a pull request, a full test-and-scan gauntlet, an automatic merge on green, a deployment, and a closed finding. Leadership reviews the trail rather than approving each step.
Observability by open standard	OpenTelemetry traces and metrics from every service into Azure Monitor, with span attributes restricted by allow-list so query text and parameters can never reach telemetry.
Deterministic synthetic data	Launch, supply-chain and telemetry data generated from a fixed seed, so expected results are exact and verifiable. Nothing proprietary is used, which is what makes the environment safe to share.

Every service below is provisioned from code, verified by an independent audit, and removed by the teardown path.

SERVICE	ROLE IN THE ENVIRONMENT	WHAT IT DEMONSTRATES
Management Groups	Subscription placed under a governed hierarchy	Governance scaffolding that survives rebuilds
Azure Policy	Tag enforcement, permitted regions, NIST SP 800-53 initiative	Compliance and cost discipline as enforced code
Microsoft Entra ID	Users, groups, Conditional Access, app registrations, managed identities, federated credentials	The identity estate as code, with no stored secrets
Microsoft Purview	Sensitivity labels, including Export-Controlled	Data classification for regulated engineering work
Microsoft Fabric	OneLake lakehouse, Delta tables, SQL analytics endpoint, data agent	A single cross-domain analytical plane
Azure Container Apps	Five applications, every one scaling to zero	Idle cost near zero without losing the platform
Azure SQL Database	Serverless with auto-pause; Entra-only authentication	An operational store with no password in existence
Azure Functions	Cost-export ingestion and token exchange	Event-driven glue on consumption pricing
Azure Key Vault	Custodian of the system's single stored secret	A deliberately minimal secret footprint
Azure Monitor	Log Analytics and Application Insights as the telemetry sink	Operational evidence the dashboards query live
Microsoft Defender for Cloud	Secure score, control breakdown, container plan toggled on demand	Security posture measured continuously and costed explicitly
Azure Cost Management	Daily exports into the lakehouse, budgets with alert thresholds	FinOps evidence rather than FinOps intention
Microsoft Copilot Studio	The custom agent, with Fabric and the tool server attached	Enterprise AI governed like any other deployed system
Azure Storage	Cost export landing zone	The first hop of the FinOps data path

ASK ANYTHING

Cross-domain copilot

A natural-language question — *which day of the week sees the most launches, and which day sees the most scrubs?* — is answered by generating and executing real SQL against the lakehouse, then returning a validated component specification the interface draws. It reaches operations, security and cost data through the same governed tool surface.

POSTURE AT A GLANCE

Control tower

Development, security and operations views framed on the Well-Architected pillars, fed by live Azure, Defender and source-control APIs plus cost exports. Vulnerabilities, secure score, compliance posture, service health and spend per application, in one place.

HANDS OFF

Self-healing pipeline

A vulnerability is found, a fix is generated, a pull request opens, the full gauntlet runs, the change merges on green and deploys, and the finding closes — unattended. The audit proves the merged commit actually reached Azure, rather than that a pull request merely existed.

The financial argument is the operational one. An environment that costs almost nothing at rest is an environment nobody is tempted to leave running, half-configured, for a quarter.

< \$5 PER MONTH, TORN DOWN	~\$1 PER HOUR, LIVE DEMO	~\$1-2 PER REBUILD CYCLE
---	--	--

\$0

IDLE AI COST

Every compute surface scales to zero, the database auto-pauses, analytics capacity is paused when idle, and AI usage is billed per interaction. The teardown path is deliberately frictionless — deleting the estate requires no approval, because making destruction easy is what keeps the rebuild honest.

WHY IT MATTERS

What a launch organization gets from this

Audit evidence as a by-product

Every layer produces a committed verification report naming the query run, the expected value, and the observed one. Demonstrating control effectiveness stops being a project and becomes an artifact of normal operation.

Export-controlled work has a place to live

Sensitivity labels are provisioned as code and persist across rebuilds, so classification is a property of the environment rather than a habit of its users.

Cloud spend attributable by construction

Resources cannot be created without the tags that make spend attributable, so cost allocation is accurate by default instead of reconstructed after the fact.

Velocity without loss of control

Automated remediation ships fixes without a human in the path, while independent verification and a small number of well-placed gates keep the boundary explicit. Speed and control are usually traded against each other; here they are separated.

Meridian Launch Systems is fictional. All data is synthetic and generated from a fixed seed; public facts such as vehicle names and launch sites are used freely, and nothing proprietary to any organization appears anywhere in the environment.